

```

import React, {
  forwardRef,
  useState,
  useRef,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import {
  ChevronDown,
  ChevronUp,
  X,
  Search,
  MoreHorizontal,
} from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
import { Thumbnail } from "@components/general/thumbnail";
import {
  CUSTOM_DROPDOWN_CONFIGS,
  DropdownColumnLayout,
} from "@components/selector/custom-dropdown-column-config";
import { Button } from "../general/button";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface ComboBoxActionButtonConfig {
  icon: React.ReactNode;
  onClick: (
    e: React.MouseEvent<HTMLButtonElement>,
    value: string | number | null,
    fullRecord: ComboBoxOption | null,
  ) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface ComboBoxOption {
  [key: string]: unknown;
  id?: string | number;
  name?: string;
}

export interface ComboBoxProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "defaultValue"
> {
  label?: string;

```

```

error?: string;
innerIcons?: Array<{ icon: React.ReactNode; className?: string }>;
actionButtons?: ComboBoxActionButtonConfig[];
prefixIcon?: React.ReactNode;
suffixIcon?: React.ReactNode;
prefixText?: string;
suffixText?: string;
value?: string | number | null;
onChange?: (
  selectedValue: string | number | null,
  fullRecordRow: ComboBoxOption | null,
) => void;
options?: ComboBoxOption[];
configKey?: string;
displayKey?: string;
valueKey?: string;
thumbnailKey?: string;
thumbnailTypeKey?: string;
placeholder?: string;
disabled?: boolean;
required?: boolean;
maxDropdownHeight?: number;
searchable?: boolean;
clearable?: boolean;
maxActionButtons?: number;
showInlineSearchRow?: boolean;
}

```

```

export const ComboBox = forwardRef<HTMLInputElement, ComboBoxProps>(
  (
    {
      label,
      error,
      innerIcons = [],
      actionButtons = [],
      prefixIcon,
      suffixIcon,
      prefixText,
      suffixText,
      value,
      onChange,
      options = [],
      configKey,
      displayKey = "name",
      valueKey = "id",
      thumbnailKey,
      thumbnailTypeKey,
      placeholder = "Select an option...",
      disabled = false,
      required = false,
    }
  )

```

```

    maxDropDownHeight = 320,
    searchable = true,
    clearable = true,
    maxActionButtons = 4,
    showInlineSearchRow = false,
    className,
    ...props
  },
  ref,
) => {
  const inputRef = useRef<HTMLInputElement>(null);
  const dropdownRef = useRef<HTMLDivElement>(null);
  const [isOpen, setIsOpen] = useState(false);
  const [searchQuery, setSearchQuery] = useState("");
  const [highlightedIndex, setHighlightedIndex] = useState(-1);
  const [selectedRecord, setSelectedRecord] = useState<ComboBoxOption | null>(
    null,
  );
  const hasError = Boolean(error);

  const config = useMemo(() : DropdownColumnLayout[] => {
    if (configKey) {
      return CUSTOM_DROPDOWN_CONFIGS[configKey] || [];
    }
    return [];
  }, [configKey]);

  const filteredOptions = useMemo(() => {
    if (!searchable || searchQuery.trim().length < 3) return options;
    const query = searchQuery.toLowerCase().trim();
    return options.filter((option: unknown) => {
      const opt = option as Record<string, unknown>;
      const displayValue =
        (opt[displayKey] as string) ||
        (opt[valueKey] as string) ||
        String(option);
      return String(displayValue).toLowerCase().includes(query);
    });
  }, [options, searchQuery, searchable, displayKey, valueKey]);

  const handleRef = useCallback(
    (node: HTMLInputElement | null) => {
      inputRef.current = node;
      if (ref) {
        if (typeof ref === "function") {
          ref(node);
        } else {
          ref.current = node;
        }
      }
    }
  );

```

```

    },
    [ref],
  );

const handleClickOutside = useCallback((event: MouseEvent) => {
  if (
    dropdownRef.current &&
    !dropdownRef.current.contains(event.target as Node)
  ) {
    if (
      inputRef.current &&
      !inputRef.current.contains(event.target as Node)
    ) {
      setIsOpen(false);
      setSearchQuery("");
      setHighlightedIndex(-1);
    }
  }
}, []);

useEffect(() => {
  document.addEventListener("mousedown", handleClickOutside);
  return () =>
    document.removeEventListener("mousedown", handleClickOutside);
}, [handleClickOutside]);

useEffect(() => {
  if (value !== undefined && options.length > 0) {
    const found = options.find((opt) => opt[valueKey] === value);
    if (found) {
      setSelectedRecord(found);
    } else if (typeof value === "string" || typeof value === "number") {
      setSelectedRecord({ [displayKey]: value, [valueKey]: value });
    }
  } else if (value === null || value === undefined || value === "") {
    setSelectedRecord(null);
  }
}, [value, options, valueKey, displayKey]);

const handleSelect = useCallback(
  (option: ComboBoxOption) => {
    const selectedValue =
      (option[valueKey] as string | number) ??
      (option[displayKey] as string | number) ??
      option;
    const fullRecord = option;
    setSelectedRecord(fullRecord);
    setSearchQuery("");
    setIsOpen(false);
    setHighlightedIndex(-1);
  },

```

```

        onChange?.(selectedValue, fullRecord);
    },
    [onChange, valueKey, displayKey],
);

const handleClear = useCallback(
    (e: React.MouseEvent<HTMLButtonElement>) => {
        e.stopPropagation();
        setSelectedRecord(null);
        setSearchQuery("");
        setIsOpen(false);
        onChange?.(null, null);
    },
    [onChange],
);

const handleKeyDown = useCallback(
    (e: React.KeyboardEvent<HTMLInputElement>) => {
        if (
            !isOpen &&
            (e.key === "ArrowDown" || e.key === "ArrowUp" || e.key === "Enter")
        ) {
            e.preventDefault();
            setIsOpen(true);
            setHighlightedIndex(0);
            return;
        }

        if (!isOpen) return;

        switch (e.key) {
            case "ArrowDown":
                e.preventDefault();
                setHighlightedIndex((prev) =>
                    Math.min(prev + 1, filteredOptions.length - 1),
                );
                break;
            case "ArrowUp":
                e.preventDefault();
                setHighlightedIndex((prev) => Math.max(prev - 1, -1));
                break;
            case "Enter":
                e.preventDefault();
                if (highlightedIndex >= 0 && filteredOptions[highlightedIndex]) {
                    handleSelect(filteredOptions[highlightedIndex]);
                }
                break;
            case "Escape":
                setIsOpen(false);
                setSearchQuery("");
        }
    },
    [isOpen, highlightedIndex, filteredOptions, handleSelect],
);

```

```

        setHighlightedIndex(-1);
        break;
    }
},
[isOpen, filteredOptions, highlightedIndex, handleSelect],
);

const limitedInnerIcons = innerIcons.slice(0, 4);
const limitedActionButtons = actionButtons.slice(0, maxActionButtons);

return (
  <div className={cn("w-full", className)}>
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
  </div>
  <div className="flex items-center w-full gap-2">
    <div
      className={cn(
        "relative flex items-center flex-1 min-w-0",
        "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
        "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
        "rounded-surface",
        "text-text-main placeholder:text-muted-foreground/60",
        "focus-within:ring-2 focus-within:ring-amber-500/20",
        "focus-within:border-amber-500 focus-within:bg-amber-500/5",
        "disabled:opacity-50 disabled:pointer-events-none",
        "disabled:cursor-not-allowed",
        "transition-all duration-200 ease-out",
        hasError &&
        "border-destructive/50 focus-within:ring-destructive/50",
        "focus-within:border-destructive/50",
        disabled && "bg-muted/50",
      )}
    >
      {(prefixIcon || prefixText) && (
        <div
          className={cn(
            "flex items-center justify-center px-3",

```

```

        "text-muted-foreground/60",
        disabled && "opacity-50",
    )}
    aria-hidden="true"
  >
    {prefixIcon && (
      <span className="w-4 h-4" aria-hidden="true">
        {prefixIcon}
      </span>
    )}
    {prefixText && (
      <span className="text-sm font-medium">{prefixText}</span>
    )}
  </div>
)}
<input
  ref={handleRef}
  type="text"
  className={cn(
    "flex-1 bg-transparent border-none outline-none",
    "px-4 py-2.5",
    "text-text-main placeholder:text-muted-foreground/60",
    "disabled:opacity-50 disabled:pointer-events-none",
    disabled:cursor-not-allowed,
    "w-full min-w-0",
    prefixIcon || prefixText ? "pl-0" : "pl-4",
    suffixIcon || suffixText || limitedInnerIcons.length > 0
      ? "pr-0"
      : "pr-4",
  )}
  disabled={disabled}
  required={required}
  placeholder={placeholder}
  value={
    selectedRecord
      ? ((selectedRecord[displayKey] as string) ??
        (selectedRecord[valueKey] as string) ??
        "")
      : ""
  }
  onClick={(e) => {
    e.stopPropagation();
    if (!disabled) setIsOpen(!isOpen);
  }}
  onKeyDown={handleKeyDown}
  readOnly
  {...props}
/>
{suffixIcon && (
  <div

```

```

        className={cn(
            "flex items-center justify-center px-3",
            "text-muted-foreground/60",
            disabled && "opacity-50",
        )}
        aria-hidden="true"
    >
        {suffixIcon}
    </div>
)}
{suffixText && (
    <div
        className={cn(
            "flex items-center justify-center px-3",
            "text-muted-foreground/60",
            disabled && "opacity-50",
        )}
        aria-hidden="true"
    >
        <span className="text-sm font-medium">{suffixText}</span>
    </div>
)}
{clearable && selectedRecord && !disabled && (
    <Button
        type="button"
        variant="ghost"
        size="icon"
        className="active:scale-95 transition-all duration-100"
        onClick={handleClear}
        aria-label="Clear selection"
    >
        <X className="w-4 h-4" aria-hidden="true" />
    </Button>
)}
<Button
    type="button"
    variant="ghost"
    size="icon"
    className="active:scale-95 transition-all duration-100 flex-shrink-0"
    disabled={disabled}
    onClick={() => setIsOpen(!isOpen)}
    aria-label={isOpen ? "Close dropdown" : "Open dropdown"}
>
    {isOpen ? (
        <ChevronUp className="w-4 h-4" aria-hidden="true" />
    ) : (
        <ChevronDown className="w-4 h-4" aria-hidden="true" />
    )}
</Button>
</div>

```



```

{limitedActionButtons.length > 0 && (
  <div className="flex items-center gap-1.5 shrink-0 ml-1.5">
    {limitedActionButtons.map((action, index) => (
      <Button
        key={index}
        variant="ghost"
        size="icon"
        disabled={disabled || action.disabled}
        onClick={e =>
          action.onClick(e, value ?? null, selectedRecord)
        }
        className="active:scale-95 transition-all duration-100"
        aria-label={action.tooltipText}
      >
        {action.icon}
      </Button>
    ))}
  </div>
)}

{isOpen && (
  <div
    ref={dropdownRef}
    className={cn(
      "absolute z-50 w-full mt-1.5",
      "bg-card/95 backdrop-blur-[var(--backdrop-blur)] border",
border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
      "rounded-surface shadow-lg",
      "max-h-[320px] overflow-auto",
      "transition-all duration-150 ease-out",
      isOpen
        ? "opacity-100 scale-100 translate-y-0"
        : "opacity-0 scale-95 -translate-y-1 pointer-events-none",
    )}
    style={{ maxHeight: maxDropdownHeight }}
    role="listbox"
    aria-label="Options"
  >
    {searchable && showInlineSearchRow && (
      <div className="p-2 border-b",
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">
        <div className="relative">
          <Search
            className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4",
text-muted-foreground/60"
            aria-hidden="true"
          />
          <input
            type="text"
            className={cn(

```

```

        "w-full bg-transparent border border-input",
        "rounded-md px-9 py-2 text-sm",
        "placeholder:text-muted-foreground/60",
        "focus:outline-none focus:ring-2 focus:ring-amber-500/20
focus:border-amber-500 focus:bg-amber-500/5",
    )}
    placeholder="Search..."
    value={searchQuery}
    onChange={(e) => {
        setSearchQuery(e.target.value);
        setHighlightedIndex(0);
    }}
    onKeyDown={(e) => e.stopPropagation()}
    autoFocus
  />
</div>
</div>
)}

{config.length > 0 && (
  <div className="px-3 py-2 bg-card/40 border-b
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] font-medium text-xs
text-muted-foreground uppercase tracking-wider">
    <div className="flex" style={{ width: "100%" }}>
      {config.map((col) => (
        <div
          key={col.key}
          className="truncate"
          style={{ width: `${col.widthPercent}%` }}
        >
          {col.headerLabel}
        </div>
      ))}
    </div>
  </div>
)}

{filteredOptions.length === 0 ? (
  <div className="px-4 py-8 text-center text-muted-foreground
text-sm">
    {searchQuery
      ? "No matching options found"
      : "No options available"}
  </div>
) : (
  <div role="listbox" aria-label="Options">
    {filteredOptions.map((option, index) => {
      const isHighlighted = index === highlightedIndex;
      const isSelected = Boolean(
        selectedRecord &&

```

```

    option[valueKey] === selectedRecord[valueKey],
  );

const renderCell = (
  col: DropdownColumnLayout,
  opt: ComboBoxOption,
) => {
  if (
    col.renderType === "thumbnail" &&
    thumbnailKey &&
    thumbnailTypeKey
  ) {
    return (
      <Thumbnail
        key={col.key}
        src={opt[thumbnailKey] as string}
        fileType={opt[thumbnailTypeKey] as string}
        size="sm"
        alt={(opt[displayKey] as string) || "Preview"}
        className="mx-auto"
      />
    );
  }
  return (
    <span key={col.key} className="truncate block">
      {(opt[col.key] as string) ??
        (opt[displayKey] as string) ??
        ""}
    </span>
  );
};

return (
  <Button
    key={(option[valueKey] as string | number) ?? index}
    type="button"
    role="option"
    aria-selected={isSelected}
    aria-highlighted={isHighlighted}
    variant="ghost"
    size="sm"
    className={cn(
      "w-full px-3 py-2 text-left text-sm",
      "transition-colors duration-100",
      "active:scale-95",
      isHighlighted && "bg-brand-primary/5 text-text-main",
      isSelected &&
        "bg-brand-primary/10 text-brand-primary font-semibold",
      disabled && "opacity-50 pointer-events-none",
    )}
  >

```

```

        onClick={() => handleSelect(option)}
        onMouseEnter={() => setHighlightedIndex(index)}
        disabled={disabled}
        style={{ maxHeight: maxDropDownHeight }}
      >
        {config.length > 0 ? (
          <div className="flex" style={{ width: "100%" }}>
            {config.map((col) => (
              <div
                key={col.key}
                className="truncate px-1"
                style={{ width: `${col.widthPercent}%` }}
              >
                {renderCell(col, option)}
              </div>
            ))}
          </div>
        ) : (
          <span className="truncate block">
            {(option[displayKey] as string) ??
              (option[valueKey] as string) ??
              String(
                option[displayKey] ?? option[valueKey] ?? "",
              )}
          </span>
        )}
      </Button>
    );
  }}}
</div>
)}

{actionButtons.length > maxActionButtons && (
  <div className="border-t
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)] p-2 bg-card/50
backdrop-blur-[var(--backdrop-blur)]">
    <Button
      type="button"
      variant="ghost"
      size="sm"
      className={cn(
        "w-full flex items-center justify-center gap-2 px-3 py-2
text-sm",
        "text-muted-foreground/70 hover:text-foreground",
        "active:scale-95 transition-all duration-100",
      )}
      onClick={() => {}}
    >
    <MoreHorizontal className="w-4 h-4" aria-hidden="true" />
    <span className="font-medium tracking-wide">

```

```

        More actions ({actionButtons.length - maxActionButtons})
      </span>
    </Button>
  </div>
)}
</div>
)}

{error && (
  <p
    className={cn(
      "mt-1.5 text-sm",
      "text-destructive/90",
      "font-medium",
    )}
    role="alert"
  >
    {error}
  </p>
)}
</div>
</div>
);
},
);

ComboBox.displayName = "ComboBox";

```